Building Odin-Orchestrator: An Engineering Journey

From a Fragile Multi-Agent Idea to a Trustworthy, Self-Verifying AI Software Company

Mostafa Adawy (Odin Markets) — June 2026

---

1. Executive summary

This document is the honest, technical story of how we built **Odin-Orchestrator**: a desktop application that runs an "AI software company" — a planning agent (the *CTO*) that breaks a goal into tasks, specialist agents (Frontend, Backend, DevOps, …) that do the work, and a verification layer that refuses to believe anyone — including itself — without proof.

It is written so that reading it top-to-bottom teaches you both *what we decided and why*, and *how the code actually works*. Every major mechanism has a matching, explained snippet in the **Appendix** (§10). The guiding value throughout was a single rule the owner set on day one and repeated often:

> **Never fake success. Never tweak a test to make it pass. A green test that was massaged is a lie; a red test that finds a real bug is the win.**

Everything below is downstream of that rule.

---

2. Main requirements (the non-negotiables)

These are the constraints we committed to before writing the orchestration. They shaped every later decision.

1. **Genuine intelligence, no hallucination.** The agent must reason, search, persist, and self-correct — and must never *claim* a result it did not actually produce.
2. **Truth in measurement.** Tests must be *real* and *un-gameable*. We fix measurement bugs; we never weaken a test to get a green.
3. **One shared context.** When a job is split, every agent must see the whole picture (the verbatim goal + the plan + everyone's work so far), not just its slice. Each new top-level prompt is treated as a **Main Task**.
4. **Production safety is sacred.** Tests run in isolated sandboxes, never on production data or paths. Public web ports (80/443) must never be taken down. Real production deploys are gated behind an explicit flag.
5. **Human in the loop where it matters.** Anything that changes files or infrastructure is proposed and approved; recurring work asks before running.
6. **The cache is earned.** Reusable scripts may be added to the library *only after* a real task proves them — never pre-filled with untested filler.

---

3. System overview (what we built)

- **Frontend:** Electron + React desktop app (`desktop/`).
- **Backend:** FastAPI + Python (`server/`).
- **Engine:** an LLM command-line tool, invoked one-shot per turn. The model was never the bottleneck — the *harness around it* was.

Core subsystems, each detailed later:

- **Main Task / shared context pool** (`main_task.py`) — the single growing history all agents read and append to.
- **Audit layer** (`audit.py`) — anti-fake-success gate + "snake-hunter" that scans green runs for hidden faults.
- **Infrastructure rotation** (`infra_rotate.py`) — firewall changes that can never lock out the public web ports.
- **Cached modules pool** (`cached_modules.py`, `cached_modules/`) — a provenance-gated library of reusable, parameterised scripts.
- **Scheduler** (`scheduler.py`) — periodic projects with an approval gate.
- **Phased acceptance gate** (`e2e_phase2_gate.py`) — one test, three phases, that must pass before promotion.

---

4. The architecture decision: one strong agent, not many

What we faced. The obvious design is many independent agents passing messages. But the literature and our own early trials agreed: most multi-agent failures are not model failures — they are *coordination and context-loss* failures. Passing summaries between agents loses detail; the parts stop composing.

What we decided in between. We researched the field (Cognition's *"Don't build multi-agents"*, Anthropic's orchestrator-workers / evaluator-optimizer, and the MAST failure taxonomy) and concluded: for *sequential, context-heavy* engineering, a **single strong agent over one shared context pool** beats naive fan-out. The "role" stops being a separate brain and becomes a **task tag** on the same agent.

How it evolved. We built `main_task.py`: a durable object seeded with the owner's verbatim prompt and the CTO's breakdown. Every step appends its result; every agent's prompt *leads* with the full pool. We added an **omission guard** (`reconcile`) that checks each extracted requirement against what was actually delivered, so "done" means *all* of it, not most. (Appendix A, B.)

---

5. The central challenge: fake success, and how we made tests ungameable

This is the heart of the project.

What we faced. An agent can write " the API returns 200 and context was shared" without having done either. Worse — *we ourselves* can make a test pass by tuning the agent to emit whatever string the test scans for. Both are the same disease: a check you can satisfy without doing the work.

A real confession. Early on, a "detail-propagation" test scanned the transcript for specific expected strings, and we made it pass by instructing the agent to echo those exact strings. That is a gamed pass. We recorded it as a failure of *our* integrity, not a success, and threw the measurement away.

What we decided. Acceptance must satisfy two properties: it accepts only when the world effect is real (*soundness*), and there must be **no fixed output the agent could emit, without doing the work, that makes the test pass** (*un-gameability*).

How it evolved — three mechanisms:

1. **Runtime-minted random tokens.** At the start of a run we mint a random token like `ODIN-7M4WJ9` (2 billion possibilities). It is introduced at one agent and must reappear, unchanged, in a *downstream* agent's output — which only receives it *through the shared pool*. Because the token is random at runtime, no pre-written transcript can contain it; reuse is possible only if context genuinely carried it. We extract by **label** (not by matching any `ODIN-` substring), so a quoted instruction can't be mistaken for the minted value — a subtlety that caused a real false positive (see §6). (Appendix C.)

2. **The snake-hunter.** A run can be "green" at the top while hiding a swallowed error, a `command not found`, or a silent timeout. The snake-hunter scans the *full* transcript for danger signatures and flags them even when the run claims success — with a small benign allow-list to avoid false alarms (e.g. the literal "0 failed" in a summary line). Fake success is flagged precisely when a success *claim* coexists with a non-empty snake list. (Appendix D.)
3. **Actions, not mentions.** We score whether an operation was *invoked and produced its effect*, never whether the transcript *talks about* it.

---

6. Infrastructure challenges (where theory met a live server)

The production outage (the hard lesson). While hardening SSH firewall rotation, a rule set defaulted to *deny* and left only port 22 open — taking the public websites (80/443) offline. The owner's diagnostic confirmed it: local services healthy, externally blocked (`UFW BLOCK … DPT=443`).

- *What we decided:* the public web ports are **sacred**. The fix had three parts: (i) restore 80/443 immediately; (ii) rewrite rotation to open 80/443 **first** and idempotently, then touch **only** port 22; (iii) codify the invariant in the code and an additive-only recovery script. (Appendix F.)
- *Why it matters:* this is why our definition of "safe" includes *continuous port reachability*, not just data integrity.

Database migrations. `prisma migrate dev` is interactive and can *reset* data on drift — fatal over SSH. We switched to `prisma migrate deploy` (additive, non-interactive) for new migrations, and learned to resolve stuck states (`P3009`) with `migrate resolve --applied`.

Shell portability. macOS ships bash 3.2 — `declare -A` (associative arrays) fails. We rewrote the deploy script with case-function config instead. An old `rsync --info=stats1` flag also broke; we removed it.

The token false-positive. A placeholder of the same shape as a real token (`ODIN-XXXXXX`) was matched by a permissive regex, so the test read the placeholder and reported *no* reuse when reuse had occurred. Fixing it (label-bound extraction) taught us that an un-gameable test must also be *precise*: a sound idea with a sloppy matcher yields both false negatives and exploitable false positives.

Database sandbox isolation. DB tests must never touch production. We create a dedicated throwaway role and database via the Postgres superuser, fingerprint production before and after, and a guard aborts unless the connection string targets the sandbox. (Appendix E.)

---

7. Phase 2: making the agent faster without lying about it

What we faced. A senior engineer reuses proven solutions instead of rebuilding from scratch. We wanted that — without polluting the system with untested "filler" scripts.

What we decided. A **provenance-gated cache** (the *Fast-Actions Library*): per-role folders of parameterised scripts with sidecar metadata (`summary`, `inputs`, `status`, `provenance`). On a new task the **role agent** (not the CTO) fetches its folder, reads the metadata, and **decides** whether a module fits — configuring and reusing it, or working from scratch. The rule: **a module is added only after it succeeds on a real task.** (Appendix G, H.)

How we proved it. An **E2E timer** records each run's wall-clock seconds and whether the cache was used; a toggle (`ODIN_DISABLE_CACHE`) lets us run a no-cache baseline, and a comparison reports the measured speedup — *from real runs only, never fabricated.* (Appendix I.)

Telling the story to the user. A top-bar **notification bell** opens two animated tabs — "The Story" (how the shared pool fixed cross-agent inconsistency) and "Caching" (what the pool is, the exact folder path, the fetch→decide→reuse flow). (Appendix N.)

---

8. Phase 3: the product layer (types, periodicity, operational agents)

What we faced. The owner wanted the system to feel like a real company: projects organised by kind, recurring work, and *non-technical* "people".

What we decided & built:

- **Project types** — a category/recommendation at creation (Web, Mobile, Integration, Deployment, Security, Operational), with a heuristic `recommend_type` default; the dashboard groups projects under these headings. It classifies; it does not change the brief. (Appendix K.)
- **Operational (non-technical) agents** — Marketing, Email, Content, Support — each with a *normal* mode (acts, still approval-gated) or *theoretical* mode (plans/drafts only). Stored on the agent as `nature` + `mode`. (Appendix L.)
- **Periodical projects + the run gate** — a project can run on a schedule, but recurring runs are **not** unattended: when due *and the device is online*, the system asks the operator to approve (run) or skip; a missed slot (device off) rolls forward silently — "no problem". (Appendix J.)

How we verified it. We extended the acceptance gate with **Phase 3**, which checks all of the above *by construction* and **without weakening Phases 1–2** (it is an added requirement, never a relaxation), and it self-cleans — creating throwaway data and deleting it. (Appendix M.)

---

9. The final state

What exists now, all compiled / parse-checked and verified:

- A **shared-context single-agent** orchestration with an omission guard.
- An **audit layer** that catches fake success and hidden faults.
- **Firewall-safe** infrastructure rotation (80/443 can never be dropped) and a sandbox-isolated DB test path.
- A **provenance-gated cache** with one proven, generic deploy/ops module, and a role-agent *fetch-and-decide* integration.
- **E2E timing** (cache vs no-cache) with honest, run-derived numbers.
- A **notification bell** with two animated explainer tabs.
- **Project types + dashboard grouping**, **operational agents**, and **periodical projects with an approval gate**.
- A single **phased acceptance gate** (Phase 1 context, Phase 2 DB-safety, Phase 3 features) where later phases never relax earlier ones.

Verified Phase-3 result (self-cleaning, real run):

```
PHASE 3 PASS = True
types   type_persisted   periodic_persisted   operational_agent
periodic_task   gate{pending  skip  missed }
leftover projects: 0 | leftover tasks: 0
```

The Phase-1/2 parts of the full gate still require the live engine + the VM to run (unchanged); Phase 3 runs standalone.

---

10. Appendix — the important code, explained

Each snippet is real code from the system, with a short note on *what it does and why it is written that way.*

Appendix A — The shared context pool (`server/main_task.py`)

The single growing history all agents read and append to. `reconcile` is the omission guard: it only lets a requirement count as "done" if enough of its key terms actually appear in the delivered work.

```
def append(pid, who, text):
    """Append to the ONE growing shared context pool - every CTO decision,
    member work, and result lands here, in order, so the whole team works from
    one continuous history (Cognition's rule)."""
    mt = get(pid)
    if not mt:
        return {}
    mt["pool"].append({"who": who, "text": text, "ts": store.now()})
    _put(mt); return mt

def pool_text(pid, limit=14000):
    """Render the full running pool - the team's single shared chat - for
    whoever is active now."""
    mt = get(pid)
    if not mt or not mt.get("pool"):
        return ""
    # … renders [who] text blocks up to `limit` chars …

def reconcile(pid, delivered_text):
    """Mark requirements addressed only if enough of their key terms appear in
    the delivered work; return the ones STILL open (the omission guard's teeth)."""
    mt = get(pid); low = (delivered_text or "").lower()
    for r in mt.get("requirements", []):
        if r["status"] == "done":
            continue
        terms = [w for w in re.findall(r"[a-zA-Z0-9_./-]{4,}", r["text"].lower())
                 if w not in ("the","and","for","with","that","this","from","into","must","should")]
        if terms and sum(1 for w in terms if w in low) >= max(1, len(terms)//2):
            r["status"] = "done"
    _put(mt)
    return [r for r in mt.get("requirements", []) if r["status"] != "done"]
```

Appendix B — Wiring the pool + cache into each agent (`server/api_v2.py`)

Every active agent's prompt *leads* with the shared pool and its role's cache catalog, so it always has the big picture and a chance to reuse proven work.

```
import main_task
_checklist = main_task.checklist_text(pid)
_pool = main_task.pool_text(pid)          # the ONE growing shared context
import cached_modules
_cached = cached_modules.index_text(member)   # this role's pool - fetch + decide

work_prompt = (f"You are the {member} agent on project '{name}'.\n{loc}\n"
               f"{_pool}\n"
               f"{_checklist}\n"
               f"{_cached}\n"
               f"YOUR MANDATE - the working spec, own EXACTLY this:\n{_mandate}\n\n"
               ...)
```

Appendix C — Un-gameable token extraction (`server/e2e_phase2_gate.py`)

Tokens are minted at runtime and extracted **by label**, so a quoted instruction can't be mistaken for the real value, and no pre-written transcript can contain the value.

```
def after(text, label):
    # find the minted token that follows an EXPLICIT label, not just any ODIN- string
    m = re.search(label + r"[^\n]{0,40}?(ODIN-[A-Z0-9]{6})", text, re.I)
    return m.group(1) if m else None

A        = after(be, "TOKEN_A")     # Backend mints it
A_in_fe  = after(fe, "USES_A")      # Frontend must echo the SAME value
reuse_A  = bool(A and A_in_fe and A_in_fe == A)   # true only if context carried it
```

Appendix D — The snake-hunter (`server/audit.py`)

Scans the full output for danger signatures an exit-0 can hide; a non-empty result means "do NOT call this clean-green," even if the claim says success.

```
_DANGER = [
    (r"\bP\d{4}\b", "Prisma migration error"),
    (r"error TS\d+", "TypeScript compile error"),
    (r"command not found|not installed|: not found", "missing tool/command"),
    (r"timed out|could not connect|connection refused", "connection failure"),
    (r"permission denied|\bEACCES\b", "permission error"),
    (r" |\bFAILED\b|\bFAIL:", "a step reported FAIL"),
    # … disk full, OOM, fatal/crash, unhandled rejection, etc. …
]

def scan_hidden_faults(text):
    out, seen = [], set()
    for line in (text or "").splitlines():
        if _BENIGN.search(line):       # e.g. "0 failed" in a summary line
            continue
        for pat, label in _DANGER:
            if re.search(pat, line, re.I):
                key = (label, line.strip()[:160])
                if key not in seen:
                    seen.add(key); out.append({"label": label, "line": line.strip()[:200]})
                break
    return out

def claims_success(text):     # did the agent ASSERT success?
    return bool(_SUCCESS.search(text or ""))
# fake success  ==  claims_success(text) AND scan_hidden_faults(text) is non-empty
```

Appendix E — Production safety: sandbox DB + fingerprint (`e2e_phase2_gate.py`)

A throwaway role+database is created via the Postgres superuser; a guard aborts unless Prisma targets the sandbox; production is fingerprinted before and after.

```
SBX_DB, SBX_ROLE = "organics_e2e_sandbox", "odin_e2e_sbx"

def db_guard():
    out = _ssh(f"... npx prisma migrate status ...").get("out","")
    safe = ('"organics_e2e_sandbox"' in out) and ('"organics"' not in out)
    return safe, out                # only proceed if we target the sandbox, NOT production

# before/after fingerprints must match, else it's a safety FAILURE
prod_before = prod_fingerprint()
# … run migrate deploy + reset on the throwaway db, then drop it …
```

```
prod_after  = prod_fingerprint()
prod_safe   = bool(prod_before) and (prod_before == prod_after)
```

Appendix F — Firewall rotation that can't drop the websites (`infra_rotate.py`)

Open 80/443 **first** and idempotently; then touch **only** port-22 rules; then re-confirm 80/443. Never flush other rules, never change the default policy in a way that drops web traffic.

```
def _ufw_script(ip):
    return (
        'set -e\n'
        f'new_ip="{ip}"\n'
        '# 1) PUBLIC WEB PORTS - always allowed (idempotent).\n'
        'sudo ufw allow 80/tcp\n'
        'sudo ufw allow 443/tcp\n'
        '# 2) SSH (22) - restrict to the single current IP, deny the rest.\n'
        "sudo ufw status numbered | awk '/22\\/tcp/ && (/ALLOW IN/ || /DENY IN/) "
        '{gsub(/\\[|\\]/,"",$2); print $2}\' | sort -rn | '
        'while read -r n; do sudo ufw --force delete "$n"; done\n'
        'sudo ufw allow from "$new_ip" to any port 22 proto tcp\n'
        'sudo ufw deny 22/tcp\n'
        '# 3) safety net: confirm 80/443 are still allowed.\n'
        'sudo ufw allow 80/tcp >/dev/null 2>&1 || true\n'
        'sudo ufw allow 443/tcp >/dev/null 2>&1 || true\n'
        'sudo ufw status numbered\n'
    )
```

Appendix G — The provenance-gated cache (`server/cached_modules.py`)

The role agent fetches its own folder + Miscellaneous and is told to *decide*. The cache can be turned off for the no-cache timing baseline.

```
def cache_enabled():
    return os.environ.get("ODIN_DISABLE_CACHE") != "1"

def list_for_role(role):
    """Modules available to this role: its own folder + Miscellaneous."""
    out = []
    for folder in [d for d in (_role_dir(role), "Miscellaneous") if d]:
        for meta in sorted((POOL/folder).glob("*.meta.json")):
            m = json.loads(meta.read_text()); m["folder"] = folder; out.append(m)
    return out

def index_text(role):
    if not cache_enabled():
        return ""                    # baseline run: hide the pool, work from scratch
    mods = list_for_role(role)
    if not mods:
        return ""
    # … renders a catalog ending with the instruction: …
    # "DECIDE: if one MATCHES your task, adapt its inputs and RUN it (faster,
    #  proven); if nothing fits, work normally. Only use status == 'working'."
```

Appendix H — The one proven module: generic deploy/ops (`cached_modules/DevOps/cloud_depl`

Configured entirely by environment variables, with the safe defaults we earned.

```
# SAFE DEFAULTS baked in: password SSH (no key prompt), secrets NEVER synced,
# firewall refresh ALWAYS preserves 80/443, migrations additive (migrate deploy
# not migrate dev), and it NEVER reboots unless REBOOT=1.
SECRET_EXCLUDES=(".env*" ".backup-env" "*.pem" "*.key"
                 "config/google-oauth-client.json" "node_modules" "dist" ".next"
                 "logs" "prisma/migrations" "package-lock.json" "uploads")
case "$MIGRATE" in
  deploy) CMDS="npx prisma generate && npx prisma migrate deploy" ;;   # additive
  reset)  CMDS="npx prisma migrate reset --force --skip-seed" ;;        # explicit only
esac
[ "$REBOOT" = "1" ] && { SSH "sudo reboot"; sleep 40; }                # never by default
```

Appendix I — E2E timing, cache vs no-cache (`server/run_timer.py`)

Start at task begin, stop at completion; persist; compute speedup from *real* recorded rows only.

```
class Timer:
    def __init__(self, pid, task, cached=None):
        self.cached = (os.environ.get("ODIN_DISABLE_CACHE") != "1") if cached is None else cached
        self.t0 = time.time()
    def stop(self, ok=True):
        secs = round(time.time() - self.t0, 2)
        rows = _load(); rows.append({"pid": self.pid, "task": self.task,
                "cached": self.cached, "ok": ok, "seconds": secs}); _save(rows)
        return secs

def compare(task_contains=""):
    rows  = [r for r in _load() if r.get("ok") and task_contains in r.get("task","")]
    cached = next((r for r in reversed(rows) if r["cached"]), None)
    plain  = next((r for r in reversed(rows) if not r["cached"]), None)
    if cached and plain and cached["seconds"] > 0:
        return {"speedup": round(plain["seconds"]/cached["seconds"], 2)}
```

Appendix J — The periodic run gate (`server/scheduler.py`)

Due + device-online + approval-required go *pending* (don't run). Missed while offline (overdue beyond a grace window) roll forward silently.

```
def _grace(schedule):
    if schedule.get("kind") == "daily":    return 2*3600.0        # act within 2h
    if schedule.get("kind") == "interval": return min(max(60.0, s*0.5), 1800.0)
    return float("inf")                                           # 'once' fires even if late

def _tick():
    now = time.time()
    for t in _all():
        if not t.get("enabled") or t.get("status") == "pending_approval":
            continue
        nr = t.get("nextRun")
        if not nr or now < nr:
            continue
        if t["schedule"]["kind"] in ("interval","daily") and now - nr > _grace(t["schedule"]):
            t["nextRun"] = _next_future(t["schedule"]); continue   # MISSED -> roll forward, no run
        if t.get("requireApproval"):
            t["status"] = "pending_approval"; t["pendingSince"] = now; continue   # ALERT, don't run
        _run_task(t); t["nextRun"] = _next_run(t["schedule"], now) # legacy auto-run path
```

```
def approve(tid):   # owner said go: run now, then reschedule
    t = get_task(tid); _run_task(t); # … set status idle, advance nextRun …
def skip(tid):      # owner said skip: advance to next slot, no run ("no problem")
    # … status idle, nextRun = next future slot …
```

Appendix K — Project types + recommendation (`server/projects.py`)

A category/recommendation, not a second brief. `recommend_type` is the smart default the picker shows.

```
PROJECT_TYPES = [
    {"key":"web","label":"Web application","icon":" ","suggests":["Frontend","Backend","QA"]},
    {"key":"mobile","label":"Mobile application","icon":" ","suggests":["Frontend","Backend","QA"]},
    {"key":"integration","label":"Integration / Scripts","icon":" ","suggests":["Backend"]},
    {"key":"deployment","label":"Deployment","icon":" ","suggests":["DevOps"]},
    {"key":"security","label":"Security","icon":" ","suggests":["Security","DevOps"]},
    {"key":"operational","label":"Operational","icon":" ","suggests":["Marketing","Email"]},
]

def recommend_type(description):
    d = " " + (description or "").lower() + " "
    if any(w in d for w in ("market","campaign","newsletter","outreach","seo ")): return "operational"
    if any(w in d for w in ("mobile","ios","android","flutter","swift")):          return "mobile"
    if any(w in d for w in ("deploy","vm","nginx","azure","hosting")):             return "deployment"
    if any(w in d for w in ("security","audit","harden","vulnerab","cve")):        return "security"
    if any(w in d for w in ("integration","webhook","etl","cron","glue ")):        return "integration"
    return "web"
```

Appendix L — Operational (non-technical) agents (`server/agents_registry.py`)

Agents now carry a `nature` (technical/operational) and a `mode` (normal = acts; theoretical = plans only).

```
def register_agent(pid, name, role="", avatar="generic", description="",
                   skills=None, restrictions=None, model="",
                   nature="technical", mode=""):
    a = { "name": name, "role": role or name, "avatar": ...,
          "nature": nature or "technical",
          "mode":   mode or ("normal" if nature == "operational" else ""),
          ... }
```

Appendix M — Phase 3 acceptance check (`server/e2e_phase2_gate.py`)

Checks types, operational-agent registration, periodical registration, and the full due→pending→approve/skip→missed gate — then deletes all test data.

```
def phase3_check():
    proj = projects.create_project(name=nm, description="weekly marketing digest",
        members=[{"name":"Marketing","role":"Marketing","nature":"operational","mode":"theoretical"}],
        periodic={"enabled":True,"schedule":{"kind":"daily","hhmm":"09:00"},"requireApproval":True})
    ag   = next(a for a in agents_registry.list_agents(proj["id"]) if a["name"]=="Marketing")
    d["operational_agent"] = ag["nature"]=="operational" and ag["mode"]=="theoretical"
    task = next(t for t in scheduler.list_tasks(proj["id"]) if t["kindTag"]=="project-periodic")

    scheduler.update_task(task["id"], nextRun=time.time()-1); scheduler._tick()
    pending_ok = scheduler.get_task(task["id"])["status"]=="pending_approval"   # due -> pending
    scheduler.skip(task["id"]);   skip_ok   = ...                               # skip -> rolls fwd
    scheduler.update_task(task["id"], nextRun=time.time()-99999); scheduler._tick()
```

```
    missed_ok  = ...                                                    # missed -> silent
    # finally: delete tasks + project so the suite never pollutes real data
```

Appendix N — Frontend: the bell + the periodic approval alert (`desktop/src/views.jsx`)

The always-on "1" badge invites the user into two animated explainer tabs; the global watcher surfaces due periodic tasks for approve/skip.

```
export function BellNotification() {
  const [open, setOpen] = useState(false);
  const [stage, setStage] = useState("invite");   // invite | details
  const [tab, setTab]   = useState("story");        // story | caching
  return (/* bell with a pulsing "1" badge → panel → The Story / Caching tabs */);
}

export function PeriodicApprovalWatcher() {
  const [pending, setPending] = useState([]);
  useEffect(() => {                                    // poll for due periodic tasks
    const tick = async () => setPending(await api.pendingSchedules());
    tick(); const h = setInterval(tick, 10000); return () => clearInterval(h);
  }, []);
  // renders a card per due task: " Approve & run"  /  "Skip this time"
}
```

---

11. Closing principle

The thread that ties every chapter together: **the same scepticism that catches a lying agent must be turned on your own tests.** We chose, repeatedly, the harder truth — a red test that exposed a real bug, an admitted gamed pass, an outage owned and fixed with an invariant — over a comfortable green. That choice is the system's real architecture.